Project Phase #2

Graded

Group

Donovan Baran
Sanjay Sriram
Daksh Agarwal
...and 1 more
View or edit group

Total Points

26.25 / 28 pts

Question 1

Common Problem Selection **3** / 3 pts

1.1 **Selected problem** **1** / 1 pt

✔ **+ 1 pt** Correct

1.2 **Selection rationale** **1** / 1 pt

✔ **+ 1 pt** Correct

1.3 **Pod agreement:** **1** / 1 pt

✔ **+ 1 pt** Correct

Question 2

Common Scope and Cost Estimate **2.75** / 3 pts

2.1 **System boundary** **1** / 1 pt

✔ **+ 1 pt** Correct

+ 0 pts Totally wrong/Empty

2.2 **Included and excluded scope** **1** / 1 pt

✔ **+ 1 pt** Correct

2.3 **Cost and feasibility estimate** **0.75** / 1 pt

✔ **+ 1 pt** Correct

💬 **– 0.25 pts** Hour estimates need ranges and more detail for each team.

Question 3

Common Specification — **2.5** / 3 pts

3.1 **Requirements** — **0.75** / 1 pt

✔ **+ 1 pt** Correct

💬 **– 0.25 pts** Security and quality requirements need more detail, including what is left out and why.

3.2 **Behavioral and information models** — **0.75** / 1 pt

✔ **+ 1 pt** Correct

💬 **– 0.25 pts** Deadline status names do not match between the diagrams and the API.

3.3 **Model explanation** — **1** / 1 pt

✔ **+ 1 pt** Correct

Question 4

Acceptance Criteria — **3** / 3 pts

4.1 **Requirement coverage** — **1** / 1 pt

✔ **+ 1 pt** Correct

4.2 **Test data and expected outputs** — **1** / 1 pt

✔ **+ 1 pt** Correct

4.3 **Independent evaluation** — **1** / 1 pt

✔ **+ 1 pt** Correct

Question 5

Cross-Team Testing Interface — **2.75** / 3 pts

5.1 **Interface contract** — **0.75** / 1 pt

✔ **+ 1 pt** Correct

💬 **– 0.25 pts** Did not say which fields are required for each report stage.

5.2 **Data schemas and examples** — **1** / 1 pt

✔ **+ 1 pt** Correct

5.3 **Repeatable test setup** — **1** / 1 pt

✔ **+ 1 pt** Correct

Question 6

Milestones for Phases 3-5 — **4** / 4 pts

6.1 **Phase 3** — **1** / 1 pt

✔ **+ 1 pt** Correct

+ 0 pts Totally wrong/Empty

6.2 **Phase 4** — **1** / 1 pt

✔ **+ 1 pt** Correct

6.3 **Phase 5** — **1** / 1 pt

✔ **+ 1 pt** Correct

6.4 **Sequencing rationale** — **1** / 1 pt

✔ **+ 1 pt** Correct

Question 7

Team System Design — **3.75** / 4 pts

7.1 **Architecture and data flow** — **0.75** / 1 pt

✔ **+ 1 pt** Correct

💬 **– 0.25 pts** Diagram shows a timer service, but the later design says it will not use one. Some API routes also do not match.

7.2 **Design decisions** — **1** / 1 pt

✔ **+ 1 pt** Correct

7.3 **Operation and observability** — **1** / 1 pt

✔ **+ 1 pt** Correct

7.4 **Validation strategy** — **1** / 1 pt

✔ **+ 1 pt** Correct

Question 8

Build Buy and Reuse — **2** / 2 pts

8.1 **Capability decisions** — **1** / 1 pt

✔ **+ 1 pt** Correct

8.2 **GenAI use** — **1** / 1 pt

✔ **+ 1 pt** Correct

Question 9

Major Risks and Investigation Plan **1.75** / 2 pts

9.1 **Priority risks** **0.75** / 1 pt

✔ **+ 1 pt** Correct

+ 0 pts Totally wrong/Empty

💬 **– 0.25 pts** Did not say who will investigate either risk.

9.2 **Risks requiring pod agreement** **1** / 1 pt

✔ **+ 1 pt** Correct

+ 0 pts Totally wrong/Empty

Question 10

References **0.75** / 1 pt

✔ **+ 1 pt** Correct

+ 0 pts Totally wrong/Empty

💬 **– 0.25 pts** References are not numbered in order of first use.

Q1 Common Problem Selection

3 Points

Shared work - complete Sections 1-6 with your pod.
Phase 2 establishes a shared specification that each team can independently implement and test. Each team submits its own report: use the same agreed content for Sections 1-6, then complete Sections 7-9 independently. Include sources in Section 10. Reuse and refine Phase 1 findings where relevant.

Q1.1 Selected problem

1 Point

(3-4 sentences): Identify the users, the task they need to accomplish, and the limitation in the current approaches. Explain how software could improve the outcome and connect the problem to specific regulatory obligations. Cite the relevant provisions.

Manufacturers of products with digital elements, particularly product security engineers and compliance personnel, need to determine which supported product releases are affected by newly discovered vulnerabilities and manage the evidence and reporting that follows. Current approaches can rely on separate SBOMs, vulnerability alerts, incident records, and manual reporting processes, making it difficult to quickly identify affected releases and keep track of regulatory deadlines. Software could connect vulnerabilities to product components and releases, organize investigation evidence, and track reporting deadlines while leaving final engineering and compliance decisions to human personnel. This supports the CRA requirement to identify and document vulnerabilities and components, including through a machine-readable SBOM under Annex I, Part II(1), as well as the Article 14(1)–(2) requirement to report actively exploited vulnerabilities, including notifications within 24 and 72 hours and a final report after corrective or mitigating measures become available.

Q1.2 Selection rationale
1 Point

(3-5 sentences): Name the Phase 1 alternatives considered and explain why the pod selected or combined these ideas. Explain why the problem involves meaningful engineering work, fits the semester, and permits independent implementations whose outputs can be compared.

The Phase 1 alternatives considered were the Automated SBOM and Vulnerability Tracking System and the CRA Article 14 Incident Reporting System. The pod combined these ideas because vulnerability identification and reporting are closely connected: the system must first determine which product releases are affected, then organize the investigation evidence and track the required reporting deadlines. The problem involves meaningful engineering work such as matching vulnerabilities to components and releases, maintaining persistent incident state, handling deadlines, and producing consistent outputs, while still being small enough to implement during the semester by reusing existing vulnerability data and tools. It also supports independent implementations because each team can design its own internal architecture while being tested with the same inputs and compared on observable outputs such as affected releases, incident status, deadlines, and generated report data.

Q1.3 Pod agreement:
1 Point

Record the meeting date, participating teams and representatives, and the agreed specification version or date. State any remaining disagreement that affects the common scope or testing behavior and how the pod will resolve it. If no disagreement remains, state that.

Meeting date: September 26, 2026

Participating teams: Team 2 and Team 6

Representatives: Chris Feng, Pataraphol Keeraswangporn, Jared Dong, Ansh Kothari, Sathvik Valluri, Daksh Agarwal, Donovan Baran, and Sanjay Sriram

Agreed specification: Common specification v1.0, agreed September 24, 2026. It covers REQ-01 through REQ-08, acceptance criteria AC-01 through AC-10, and the Section 5 web-service interface.

Remaining disagreements: One disagreement affects common testing behavior: whether Article 14 "awareness" starts at the raw alert time or at engineer-verified confirmation. The pod resolved this for testing by treating awareness_time as a user-supplied field (REQ-05). The system never infers awareness, so both teams produce identical deadlines from the same inputs. Any legal interpretation is outside the common scope and is tracked as Risk 2 in Section 9.

Two other differences remain, but neither affects common scope or testing, because both teams are tested only through the Section 5 interface:

Whether to bind SBOMs to shipped releases or generate them live in CI/CD. Whether to track deadlines with a workflow engine (Temporal or TheHive) or compute them from stored timestamps.

Each team will decide these in its own design.

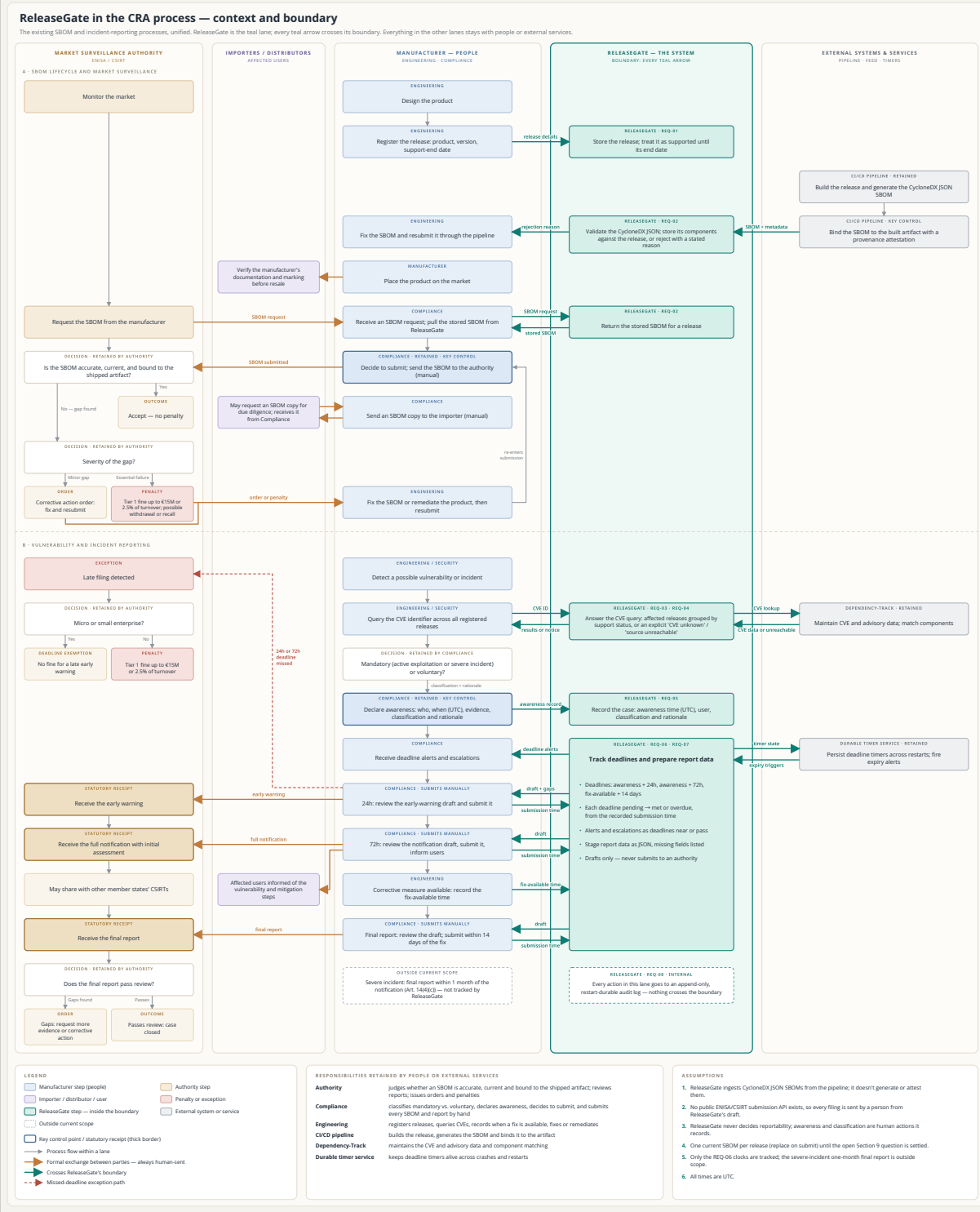
Q2 Common Scope and Cost Estimate
3 Points

Shared work - complete with your pod.

Q2.1 System boundary
1 Point

Insert a context or boundary diagram showing the common system, users, external systems, and the main information exchanged across the boundary. Label responsibilities retained by people or external services. Use any clear notation; identify assumptions the diagram relies on.

2.1.svg Download



Q2.2 Included and excluded scope
1 Point

(3-5 bullets each): State the capabilities every team will implement and the capabilities explicitly ***excluded***. Identify limits on supported data, users, formats, or operating conditions where they constrain the project. Reference the regulatory obligations the included capabilities support.

Included:
1. Release registration with a support-end date (REQ-01; Art. 13(8))

2. CycloneDX JSON SBOM upload, validation and rejection with a stated reason (REQ-02; Annex I, Part II(1))

3. CVE lookup grouped by support status, with an explicit message when the CVE is unknown or the source is unreachable (REQ-03, REQ-04; Annex I, Part II(1)–(2))

4. Incident cases with awareness time and classification, plus tracking of the 24h, 72h and 14-day deadlines (REQ-05, REQ-06; Art. 14(1)–(2), Art. 15)

5. Report data for each stage as JSON, and an append-only audit log (REQ-07, REQ-08; Art. 14(2), Art. 16)

Excluded:
1. Generating or attesting SBOMs (done by the CI/CD pipeline)

2. Submitting to ENISA, CSIRTs, authorities or importers (a person does this by hand)

3. Deciding whether an incident is legally reportable (a human decision that ReleaseGate records)

4. Maintaining the vulnerability database (Dependency-Track does this)

5. The one-month final report for severe incidents (Art. 14(4)(c))

Limits: CycloneDX JSON is the only accepted format, Dependency-Track is the only CVE source, users are internal staff only, all times are UTC, and recall is tested only against the seeded dataset.

Q2.3 Cost and feasibility estimate
1 Point

Estimate the cost (number of person-hours) for each team to deliver this scope through Phase 6. Break the estimate into major work areas; include design, implementation, integration, validation, and deployment. State the available GenAI tools or licenses, expected use, and how they affect the estimate. Use ranges where uncertain; explain differences between teams.

1. Design: 2 h per person (8 h team), week 1. Data model, deadline rules and the report fields for each stage.

2. Implementation: 5 h per person (20 h team), weeks 1–3. REQ-01 to REQ-08.

3. Integration: 2 h per person (8 h team), week 3. Dependency-Track, the timer service and SBOM intake from the pipeline.

4. Validation: 2 h per person (8 h team), week 4. Recall on the seeded dataset, deadline edge cases, rejection reasons and log immutability.

5. Deployment: 1 h per person (4 h team), week 4. Environment setup and a short runbook.

6. GenAI: Every team is assumed to have a coding assistant (for example Claude Code or Copilot) that writes most of the boilerplate code and tests. Without it this scope would take roughly 3–4 times longer. People still write and check the deadline logic and the audit log.

7. Risk: 12 hours per person is tight. If a team falls behind, cut the integration work first: replace the external timer service with local persistent timers and use a local copy of the CVE data.

8. Differences between teams: Teams that already know CycloneDX or Dependency-Track will need fewer implementation hours. Teams new to timers or persistent storage should move hours from design into integration.

Q3 Common Specification

3 Points

Shared work - complete with your pod.

Q3.1 Requirements
1 Point

List every consequential requirement that each implementation must satisfy. Assign stable identifiers (REQ-01, REQ-02, and so on) and reuse them in later sections. Write each requirement as an observable behavior or a checkable constraint, including applicable conditions. Cite its regulatory source or explain the stakeholder need it addresses.
Be careful about scope. If there are only a few requirements, you are likely missing important capability. If your list is too long, each requirement may be too narrowly written, or your spec may be too ambitious for a semester project. Cover the main successful behavior, invalid inputs and failures, important data or state, and applicable security, regulatory, and quality requirements. State measurable limits where needed. Identify categories that do not apply and explain why. Interface details may be defined once in Section 5 and referenced here.
Wording: Choose a pattern that fits. The examples below are optional. (Eg. For state-dependent behavior, use “While [state], the system shall [response].”) See the EARS resource for other patterns: https://alistairmavin.com/ears/

REQ-01 — When a user registers a release with a product, version, and support-period end date, the system shall store it and treat it as supported until that date.
Source: Art. 13(8).
REQ-02 — When a CycloneDX JSON SBOM is submitted for a registered release, the system shall store its components against that release. The system shall reject invalid files with an error that states the reason.
Source: Annex I, Part II(1).
REQ-03 — When a user queries a Common Vulnerabilities and Exposures (CVE) identifier, the system shall return every release containing an affected component, grouped by support status, with 100% recall on the pod's seeded test dataset.
Source: Annex I, Part II(1)–(2).
REQ-04 — If the CVE is unknown or the vulnerability source is unreachable, the system shall say so explicitly rather than return an empty result.
Source: stakeholder need, since an empty result would give false assurance.
REQ-05 — When a user opens an incident case, the system shall record the awareness time (UTC), the user, and a mandatory or voluntary classification with a rationale.
Source: Art. 14(1); Art. 15.
REQ-06 — For a mandatory case, the system shall compute deadlines at awareness + 24 hours, awareness + 72 hours, and fix-available + 14 days. It shall show each deadline as pending, met, or overdue based on recorded submission times.
Source: Art. 14(2)(a)–(c).
REQ-07 — When a user requests report data for a stage, the system shall output JSON with that stage's fields (Section 5), listing any missing fields.

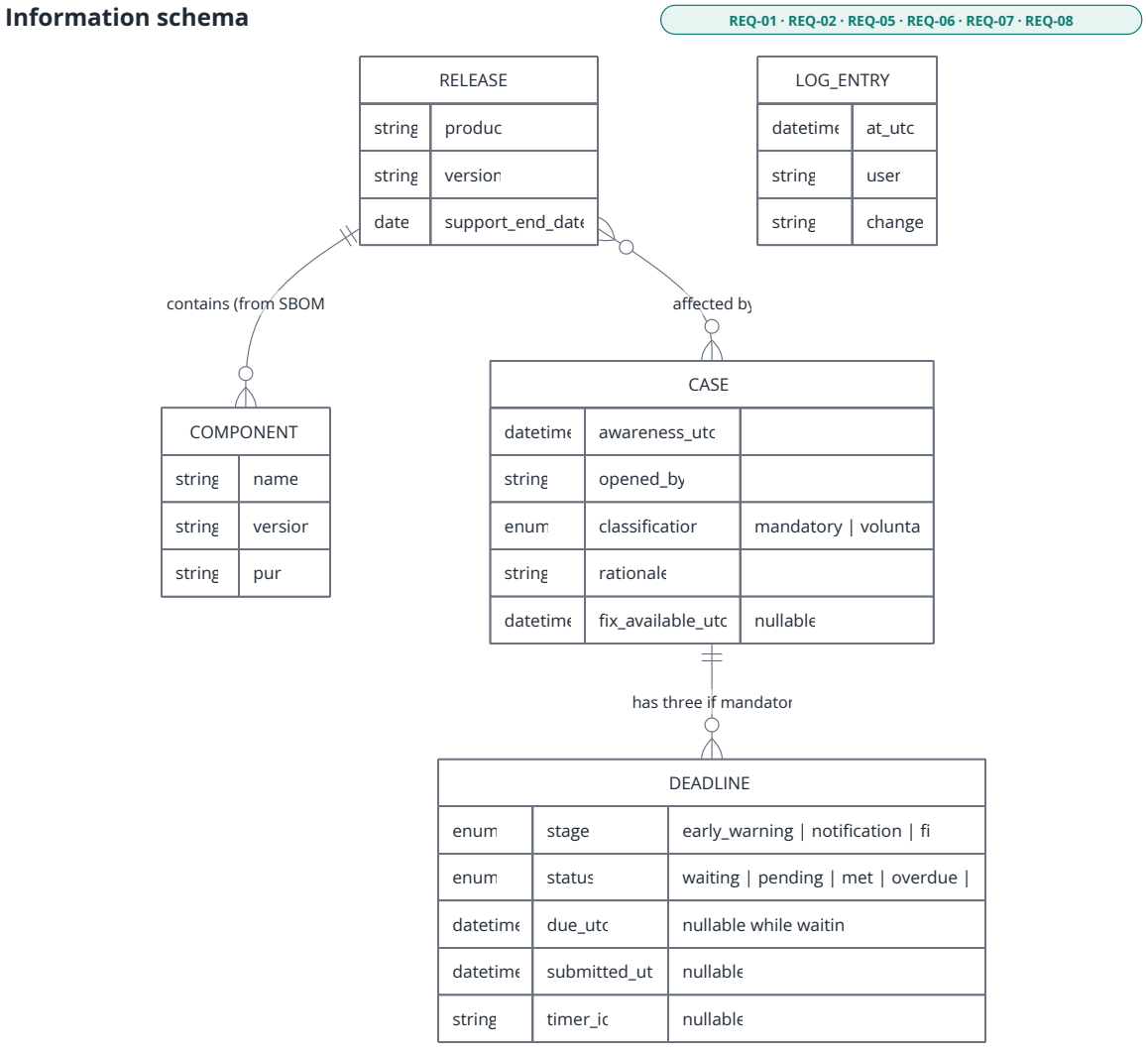
Source: Art. 14(2); Art. 16.

REQ-08 — The system shall log every change with its time and user, shall not allow log entries to be edited, and shall retain all data across restarts.
Source: Art. 14 evidence requirements.

Q3.2 Behavioral and information models
1 Point

Provide models precise enough to guide independent implementations. Show the main behavior, important data or state, and relevant failure paths. Suitable models include workflows, state machines, schemas, invariants, or structured examples. Use the views your system needs; do not reproduce the same information in multiple forms. Beside each model, list the requirement identifiers from Section 3.1 that it helps describe or clarify.

3.1_information.svg Download



INVARIANTS

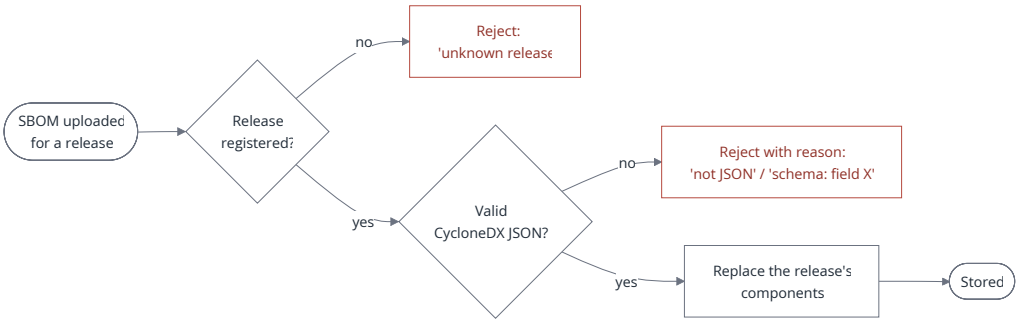
- Support status is derived: supported ⇔ today ≤ support_end_date. It is never stored.
- A release holds the components of its latest valid SBOM only (replace on upload; see Section 9).
- A mandatory case gets three DEADLINE rows. Early warning and notification start pending with timers set;
- the final report starts waiting until fix_available_utc is recorded, then its timer is set.
- DEADLINE.status changes only on a timer firing or a submission. A submission cancels the timer.
- LOG_ENTRY is append-only: no update or delete path exists. One entry per change; all data survives restart.
- Stage report output = that stage's Section 5 fields as JSON, plus a missing_fields list naming every empty one.

3.1_behavior.svg

Download

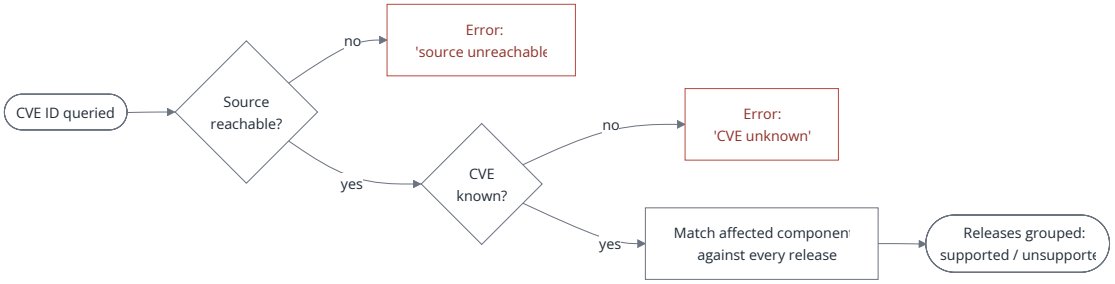
SBOM upload

REQ-02



CVE query

REQ-03 · REQ-04



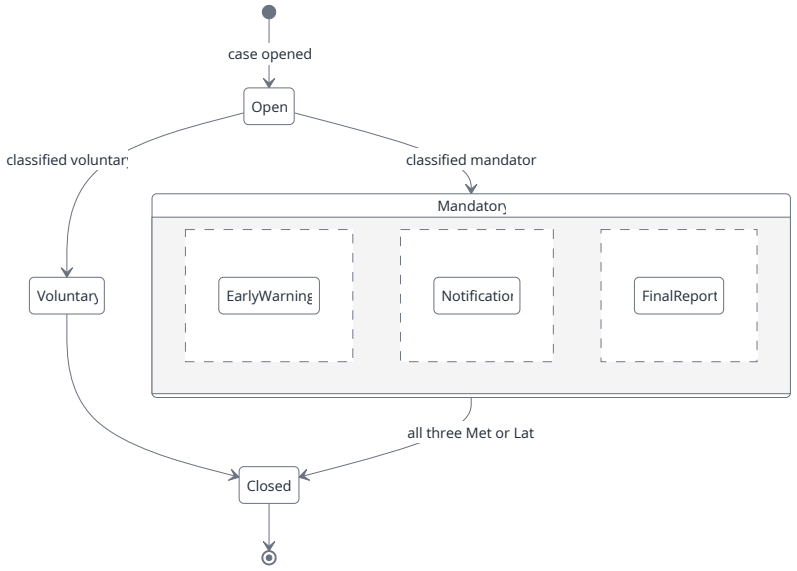
RULE

- An empty result is never returned for a failure. Recall target: 100% on the seeded dataset.

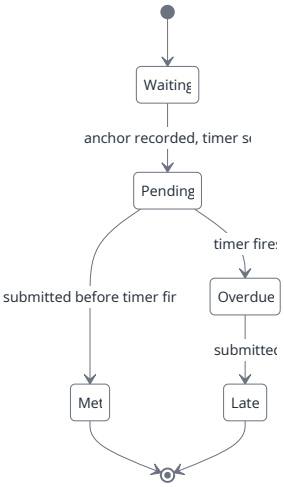
Incident case and deadlines

REQ-05 · REQ-06

CASE



EACH DEADLINE (x3, CONCURRENT)



DEADLINE	TIMER SET WHEN	TIMER FIRES AT
Early warning	case opens	awareness + 24 h
Notification	case opens	awareness + 72 h
Final report	fix-available recorded	fix-available + 14 days

RULES

- Timers run in the durable timer service, so they survive a ReleaseGate restart.
- A timer firing moves Pending to Overdue and sends an alert to Compliance and Engineering.
- Recording a submission cancels that deadline's timer. Met and Late are both final.
- On startup, any Pending deadline without a live timer gets a new one.

Q3.3 Model explanation
1 Point

(4-5 sentences): Explain how your models fit together, and clarify any relationships or decisions that may not be so obvious when looking at your models. Additionally, explain any assumptions that, if left open, could cause teams to implement different behavior. (Record significant unresolved questions in Section 9.)

The two files fit together as behavior and the data it works on. The schema fixes what gets stored and the behavior models show how each requirement moves through that data including what happens when something fails. Deadlines work like alarm clocks. When a mandatory case opens, the system stores a deadline row for each of the three reports and sets a timer in an outside timer service for the early warning and the notification, while the final report's timer waits until someone records that a fix is available. A deadline's status only changes when its timer goes off, which marks it overdue, or when a submission is recorded, which marks it met or late and cancels the timer. Because the timers live outside ReleaseGate, the system checks on startup that every pending deadline still has a live timer, so a crash can never quietly lose one. Support status works the other way and is never stored, since it's simply today's date compared with the support end date. One open assumption is that a release keeps only its latest SBOM and a new upload replaces the old one, which is logged in Section 9 for the pod to confirm. Another is that the CVE query always checks whether the source is reachable before checking whether the CVE is known, so an outage can never be reported as a CVE that doesn't exist.

Q4 Acceptance Criteria

3 Points

Shared work - complete with your pod.

Q4.1 Requirement coverage
1 Point

For every requirement in Section 3, define evidence and a pass/fail rule. Assign acceptance-criterion identifiers (AC-01, AC-02, and so on) and list the requirement identifiers they evaluate. One criterion may cover multiple requirements, and one requirement may need multiple criteria.
For each criterion, specify the setup or input, action or check, expected result, and method of evaluation. Include relevant failure or boundary cases. Quantify thresholds and tolerances when they affect acceptance. Identify which checks can run automatically and which require inspection, analysis, or human judgment.

AC-01 — Evaluates REQ-01. Given a fixed reference clock, an automated test submits four release registrations: one with a past support end date, one matching the current reference date, one with a future date, and one missing a support end date. The system passes if stored records classify the current and future releases as supported through the end of the boundary day, classify the past release as unsupported, and reject the incomplete input without saving; otherwise, it fails.

AC-02 — Evaluates REQ-02. Given two registered releases of a product, an automated test submits a valid 25-component CycloneDX JSON SBOM for the first release and queries both inventories. The system passes if stored records return 100% of the 25 components with matching names and versions under the first release and zero components under the second; otherwise, it fails.

AC-03 — Evaluates REQ-02. Given a registered release, the evaluator submits malformed JSON, JSON missing required CycloneDX fields, plaintext, and a valid SBOM targeting an unregistered release. The system passes if automated checks confirm all four inputs are rejected with zero components stored, and human inspection verifies each error message states the specific failure reason; otherwise, it fails.

AC-04 — Evaluates REQ-03. Given the pod's seeded dataset of supported and unsupported releases with known vulnerable and patched components, an automated script queries five test CVEs. The system passes if every query completes within 10 seconds and matches the ground-truth manifest with 100% recall, 100% precision, and accurate support-status grouping; otherwise, it fails.

AC-05 — Evaluates REQ-04. Given the seeded dataset, an automated fault-injection test queries a valid CVE affecting no releases, an unknown CVE, and a valid CVE while the vulnerability source is unreachable. The system passes if the first returns zero affected releases while the second and third return explicit unknown-CVE and unreachable-source errors within 15 seconds

rather than empty lists; otherwise, it fails.

AC-06 — Evaluates REQ-05. Given valid mandatory and voluntary incident submissions with UTC awareness times and rationales alongside an invalid submission missing a rationale or valid classification, the evaluator submits all three and reads back the stored records. The system passes if automated checks confirm the invalid request is rejected and valid cases preserve the user, classification, and UTC timestamp within plus or minus 2 seconds, and human inspection verifies the rationale text is intact; otherwise, it fails.

AC-07 — Evaluates REQ-06. Given a mandatory incident with fixed awareness and fix-available times, an automated clock-injection test checks initial deadlines, records an early warning at the exact 24-hour boundary, advances 1 second past the 72-hour deadline without a notification, and records a final report 13 days after the fix. The system passes if deadlines are computed at plus 24 hours, plus 72 hours, and plus 14 days with 0-second tolerance, and statuses transition from pending to met for the early warning and final report and overdue for the notification; otherwise, it fails.

AC-08 — Evaluates REQ-07. Given a complete mandatory incident and an incomplete incident missing two required 72-hour stage fields, the evaluator requests JSON reports for all stages. The system passes if automated schema checks and human inspection confirm all outputs are valid Section 5 JSON, with the complete case containing 100% of stage fields and an empty missing-fields list and the incomplete case listing the exact two missing fields; otherwise, it fails.

AC-09 — Evaluates REQ-08. Given an active system where a user registers a release, submits an SBOM, opens an incident, and records a submission, the evaluator queries the audit log and attempts to edit or delete an entry. The system passes if automated tests confirm all four actions are logged chronologically with the user and UTC timestamp within plus or minus 2 seconds while rejecting edits, and code inspection confirms no update or delete paths exist; otherwise, it fails.

AC-10 — Evaluates REQ-01, REQ-02, REQ-05, REQ-06, and REQ-08. Given a system populated with releases, SBOM components, an active mandatory incident, and audit logs, an automated test captures a state snapshot, restarts the process, and re-queries all records. The system passes if the post-restart state matches the pre-restart snapshot with 100% data retention and 0-second timestamp drift; otherwise, it fails.

Q4.2 Test data and expected outputs
1 Point

(3-4 sentences): Explain where representative valid, invalid, and boundary-case data will come from and how the pod will establish trustworthy expected outputs. Explain how the evaluation will remain repeatable if external services, changing datasets, or nondeterministic AI behavior are involved.

The pod will create a shared test dataset with sample product releases, valid CycloneDX JSON SBOMs, known vulnerability records, and mandatory and voluntary incident cases, plus invalid files, missing fields, unknown CVEs, and simulated service failures. Boundary cases will cover support expiration, submissions just before, at, and after deadlines, missing fix dates, and month and year changes. Before testing, both teams will check the expected affected releases, support status, deadlines, report fields, and error messages against the test data, Section 5 formats, and relevant CRA provisions. Tests will use the same saved vulnerability data, simulated service responses, and controlled UTC clock so results remain repeatable without relying on live services or AI-generated answers.

Q4.3 Independent evaluation
1 Point

(2-3 sentences): Explain which criteria another team can evaluate through the common web service interface (which we will define in section 5) without knowing your internal design. For criteria requiring other forms of evidence, identify the artifact and how it will be checked. A credible evaluation plan is required here; completed tests and results are not yet required.

Another team can evaluate AC-01 through AC-04 and AC-06 through AC-08 entirely through the Section 5 web service interface without knowing our internal design by running automated HTTP checks against the shared dataset and controlled UTC clock, paired with human inspection of the returned JSON response artifacts to verify error reasons (AC-03), rationales (AC-06), and stage-report fields (AC-08). AC-05 and AC-10 also send requests and check outputs through the web interface, but require the evaluating team to use the test-runner environment to stop the mock vulnerability service for fault injection (AC-05) and to restart the application process before running an automated diff on pre- and post-restart JSON state snapshot artifacts (AC-10). Finally, while AC-09 uses the web interface to confirm that state changes are logged and edit requests fail, verifying true append-only immutability requires inspecting the team's database schema and persistence-layer source code artifacts to confirm that no update or delete operations exist on the audit store.

Q5 Cross-Team Testing Interface

3 Points

Shared work - complete with your pod.

Define the common web-service interface that every team will provide. Automated tests must be able to send requests to any team's deployed service and check the observable results. A network-accessible implementation is required beginning with Phase 3.

Q5.1 Interface contract
1 Point

Provide an OpenAPI document or an equivalent structured HTTP specification covering every operation needed for cross-team testing. For each operation, state its purpose and related requirement identifiers, HTTP method and path, request format, successful response, and error responses with status codes. Define relevant state changes and behavior needed to interpret results.

The pod will use a REST-style HTTP API with JSON request and response bodies. All timestamps are ISO 8601 UTC timestamps ending in Z, and all mutating application requests require an X-User-Id header so that changes can be attributed in the audit log. Authentication is outside the common testing scope; X-User-Id identifies the acting test user but is not an authentication mechanism. The interface exposes only behavior needed to test the common specification and does not constrain either team's internal database, framework, vulnerability scanner, workflow engine, or user interface.

```
openapi: 3.1.0
info:
  title: CRA Vulnerability Response Service
  version: 1.0.0

servers:
  - url: http://{host}:{port}
    variables:
      host:
        default: localhost
      port:
        default: "8080"

paths:

  /api/v1/releases:
    post:
      summary: Register a product release
      description: >
        Stores a product release and its support-period end date.
        A release is supported through the end of support_end_date.
      x-requirements: [REQ-01, REQ-08]
      parameters:
        - $ref: "#/components/parameters/UserId"
      requestBody:
        required: true
        content:
```

```
      application/json:
        schema:
          $ref: "#/components/schemas/ReleaseCreate"
    responses:
      "201":
        description: Release registered
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/Release"
      "400":
        description: Missing or invalid input
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/Error"
      "409":
        description: Product/version already registered
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/Error"

  /api/v1/releases/{releaseId}:
    get:
      summary: Retrieve a registered release
      x-requirements: [REQ-01]
      parameters:
        - $ref: "#/components/parameters/ReleaseId"
      responses:
        "200":
          description: Release record
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/Release"
        "404":
          description: Release not found
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/Error"

  /api/v1/releases/{releaseId}/sbom:
    put:
      summary: Submit or replace the current CycloneDX JSON SBOM
```

```
    description: >
      The SBOM must be valid CycloneDX JSON and belong to an already
      registered release. Invalid input must not modify stored components.
    x-requirements: [REQ-02, REQ-08]
    parameters:
      - $ref: "#/components/parameters/ReleaseId"
      - $ref: "#/components/parameters/UserId"
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            description: CycloneDX JSON document
    responses:
      "200":
        description: SBOM accepted and stored
        content:
          application/json:
            schema:
              type: object
              required: [release_id, component_count]
              properties:
                release_id:
                  type: string
                component_count:
                  type: integer
                  minimum: 0
      "400":
        description: >
          Malformed JSON or invalid CycloneDX document. The response must
          state the reason and no components may be stored from the request.
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/Error"
      "404":
        description: Release not found
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/Error"

  /api/v1/releases/{releaseId}/components:
    get:
      summary: Retrieve components stored for a release
```

```
    x-requirements: [REQ-02]
    parameters:
      - $ref: "#/components/parameters/ReleaseId"
    responses:
      "200":
        description: Component inventory; empty array if no SBOM has been
stored
        content:
          application/json:
            schema:
              type: object
              required: [release_id, components]
              properties:
                release_id:
                  type: string
                components:
                  type: array
                  items:
                    $ref: "#/components/schemas/Component"
      "404":
        description: Release not found
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/Error"

  /api/v1/vulnerabilities/{cveId}/affected-releases:
    get:
      summary: Find releases affected by a CVE
      description: >
        Returns affected releases grouped by current support status.
        A known CVE that affects no stored release returns HTTP 200 with
        empty supported and unsupported arrays.
      x-requirements: [REQ-03, REQ-04]
      parameters:
        - name: cveId
          in: path
          required: true
          schema:
            type: string
            pattern: "^CVE-[0-9]{4}-[0-9]+$"
      responses:
        "200":
          description: CVE is known; affected releases returned
          content:
            application/json:
```

```
          schema:
            $ref: "#/components/schemas/AffectedReleaseResult"
      "404":
        description: CVE is unknown to the configured vulnerability source
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/Error"
            example:
              error:
                code: CVE_UNKNOWN
                message: CVE identifier is not known
      "503":
        description: Vulnerability source cannot be reached
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/Error"
            example:
              error:
                code: VULNERABILITY_SOURCE_UNREACHABLE
                message: Vulnerability information is currently unavailable

  /api/v1/incidents:
    post:
      summary: Open an incident case
      description: >
        Records organizational awareness and the user's mandatory or
        voluntary classification. Mandatory incidents immediately receive
        24-hour and 72-hour deadlines.
      x-requirements: [REQ-05, REQ-06, REQ-08]
      parameters:
        - $ref: "#/components/parameters/UserId"
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/IncidentCreate"
      responses:
        "201":
          description: Incident created
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/Incident"
```

```
      "400":
        description: >
          Invalid classification, missing rationale, or invalid UTC
          awareness timestamp
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/Error"

  /api/v1/incidents/{incidentId}:
    get:
      summary: Retrieve incident state and calculated deadlines
      x-requirements: [REQ-05, REQ-06]
      parameters:
        - $ref: "#/components/parameters/IncidentId"
      responses:
        "200":
          description: Incident record
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/Incident"
        "404":
          description: Incident not found
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/Error"

    patch:
      summary: Record when a corrective or mitigating measure became
available
      description: >
        Setting fix_available_at causes the final-report deadline for a
        mandatory vulnerability case to become fix_available_at + 14 days.
      x-requirements: [REQ-06, REQ-08]
      parameters:
        - $ref: "#/components/parameters/IncidentId"
        - $ref: "#/components/parameters/UserId"
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [fix_available_at]
```

```
          properties:
            fix_available_at:
              type: string
              format: date-time
    responses:
      "200":
        description: Incident updated and final deadline calculated
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/Incident"
      "400":
        description: Invalid timestamp
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/Error"
      "404":
        description: Incident not found
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/Error"

  /api/v1/incidents/{incidentId}/submissions:
    post:
      summary: Record a regulatory-report submission time
      description: >
        Records when a report stage was submitted and recomputes that
        deadline's status.
      x-requirements: [REQ-06, REQ-08]
      parameters:
        - $ref: "#/components/parameters/IncidentId"
        - $ref: "#/components/parameters/UserId"
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [stage, submitted_at]
              properties:
                stage:
                  $ref: "#/components/schemas/ReportStage"
                submitted_at:
                  type: string
```

```
            format: date-time
    responses:
      "201":
        description: Submission recorded
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/Incident"
      "400":
        description: Invalid stage or timestamp
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/Error"
      "404":
        description: Incident not found
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/Error"

  /api/v1/incidents/{incidentId}/report-data/{stage}:
    put:
      summary: Add or replace data used to generate a report stage
      description: >
        Partial report data is allowed so that the report-generation
        operation can identify fields that are still missing.
      x-requirements: [REQ-07, REQ-08]
      parameters:
        - $ref: "#/components/parameters/IncidentId"
        - $ref: "#/components/parameters/Stage"
        - $ref: "#/components/parameters/UserId"
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/ReportData"
      responses:
        "200":
          description: Report data stored
        "400":
          description: Invalid stage or field format
          content:
            application/json:
              schema:
```

```
            $ref: "#/components/schemas/Error"
      "404":
        description: Incident not found
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/Error"

  /api/v1/incidents/{incidentId}/reports/{stage}:
    get:
      summary: Generate structured data for a CRA report stage
      description: >
        Returns the stored and system-generated fields for the requested
        stage together with an explicit missing_fields array.
      x-requirements: [REQ-07]
      parameters:
        - $ref: "#/components/parameters/IncidentId"
        - $ref: "#/components/parameters/Stage"
      responses:
        "200":
          description: Report data generated
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/GeneratedReport"
        "404":
          description: Incident not found
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/Error"

  /api/v1/audit-logs:
    get:
      summary: Retrieve the append-only audit log
      x-requirements: [REQ-08]
      responses:
        "200":
          description: Audit events in chronological order
          content:
            application/json:
              schema:
                type: object
                required: [events]
                properties:
                  events:
```

```
            type: array
            items:
              $ref: "#/components/schemas/AuditEvent"

/api/v1/audit-logs/{eventId}:
  parameters:
    - name: eventId
      in: path
      required: true
      schema:
        type: string

  patch:
    summary: Audit entries cannot be modified
    x-requirements: [REQ-08]
    responses:
      "405":
        description: Audit log is immutable

  delete:
    summary: Audit entries cannot be deleted
    x-requirements: [REQ-08]
    responses:
      "405":
        description: Audit log is immutable

/api/v1/test/reset:
  post:
    summary: Reset system state for repeatable cross-team testing
    description: >
      TEST DEPLOYMENTS ONLY. Removes releases, SBOMs, incidents,
      submissions, report data, and audit events and restores the
      test clock and vulnerability fixture state.
    responses:
      "204":
        description: Test state reset

/api/v1/test/clock:
  put:
    summary: Set the reference UTC clock used during automated tests
    description: TEST DEPLOYMENTS ONLY.
    requestBody:
      required: true
      content:
        application/json:
          schema:
```

```
          type: object
          required: [now]
          properties:
            now:
              type: string
              format: date-time
    responses:
      "200":
        description: Test clock set
        content:
          application/json:
            schema:
              type: object
              required: [now]
              properties:
                now:
                  type: string
                  format: date-time

  /api/v1/test/vulnerability-source:
    put:
      summary: Configure deterministic vulnerability data or simulate failure
      description: >
        TEST DEPLOYMENTS ONLY. Allows both teams to run the same seeded
        CVE tests and simulate an unavailable vulnerability source without
        constraining the production vulnerability-data implementation.
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [mode]
              properties:
                mode:
                  type: string
                  enum: [available, unreachable]
                records:
                  type: array
                  items:
                    $ref: "#/components/schemas/VulnerabilityFixture"
      responses:
        "200":
          description: Test vulnerability source configured

components:
```

```
parameters:

  UserId:
    name: X-User-Id
    in: header
    required: true
    schema:
      type: string
      minLength: 1

  ReleaseId:
    name: releaseId
    in: path
    required: true
    schema:
      type: string

  IncidentId:
    name: incidentId
    in: path
    required: true
    schema:
      type: string

  Stage:
    name: stage
    in: path
    required: true
    schema:
      $ref: "#/components/schemas/ReportStage"

schemas:

  ReleaseCreate:
    type: object
    required: [product, version, support_end_date]
    properties:
      product:
        type: string
      version:
        type: string
      support_end_date:
        type: string
        format: date
```

```
Release:
  type: object
  required:
    [release_id, product, version, support_end_date, support_status]
  properties:
    release_id:
      type: string
    product:
      type: string
    version:
      type: string
    support_end_date:
      type: string
      format: date
    support_status:
      type: string
      enum: [supported, unsupported]

Component:
  type: object
  required: [name, version]
  properties:
    name:
      type: string
    version:
      type: string
    purl:
      type: [string, "null"]

AffectedRelease:
  type: object
  required:
    [release_id, product, version, support_end_date, matched_components]
  properties:
    release_id:
      type: string
    product:
      type: string
    version:
      type: string
    support_end_date:
      type: string
      format: date
    matched_components:
      type: array
      items:
```

```
        $ref: "#/components/schemas/Component"

  AffectedReleaseResult:
    type: object
    required: [cve_id, supported, unsupported]
    properties:
      cve_id:
        type: string
      supported:
        type: array
        items:
          $ref: "#/components/schemas/AffectedRelease"
      unsupported:
        type: array
        items:
          $ref: "#/components/schemas/AffectedRelease"

  IncidentCreate:
    type: object
    required: [awareness_at, classification, rationale]
    properties:
      cve_id:
        type: [string, "null"]
      awareness_at:
        type: string
        format: date-time
      classification:
        type: string
        enum: [mandatory, voluntary]
      rationale:
        type: string
        minLength: 1

  Deadline:
    type: object
    required: [status]
    properties:
      deadline_at:
        type: [string, "null"]
        format: date-time
      submitted_at:
        type: [string, "null"]
        format: date-time
      status:
        type: string
        enum: [pending, met, overdue, not_applicable]
```

```yaml
Incident:
  type: object
  required:
    - incident_id
    - awareness_at
    - created_by
    - classification
    - rationale
    - deadlines
  properties:
    incident_id:
      type: string
    cve_id:
      type: [string, "null"]
    awareness_at:
      type: string
      format: date-time
    created_by:
      type: string
    classification:
      type: string
      enum: [mandatory, voluntary]
    rationale:
      type: string
    fix_available_at:
      type: [string, "null"]
      format: date-time
    deadlines:
      type: object
      required:
        [early_warning, vulnerability_notification, final_report]
      properties:
        early_warning:
          $ref: "#/components/schemas/Deadline"
        vulnerability_notification:
          $ref: "#/components/schemas/Deadline"
        final_report:
          $ref: "#/components/schemas/Deadline"

ReportStage:
  type: string
  enum:
    - early_warning
    - vulnerability_notification
    - final_report
```

```
ReportData:
  type: object
  description: >
    Partial data is accepted. Fields not yet supplied are reported
    through missing_fields when the report is generated.
  properties:
    member_states:
      type: array
      items:
        type: string

    product_information:
      type: string
    exploit_nature:
      type: string
    vulnerability_nature:
      type: string
    corrective_measures_taken:
      type: string
    user_mitigations:
      type: string
    sensitivity:
      type: string

    vulnerability_description:
      type: string
    severity:
      type: string
    impact:
      type: string
    malicious_actor_information:
      type: string
    corrective_measure_details:
      type: string

GeneratedReport:
  type: object
  required:
    [incident_id, stage, generated_at, data, missing_fields]
  properties:
    incident_id:
      type: string
    stage:
      $ref: "#/components/schemas/ReportStage"
    generated_at:
```

```
      type: string
      format: date-time
    data:
      $ref: "#/components/schemas/ReportData"
    missing_fields:
      type: array
      items:
        type: string

AuditEvent:
  type: object
  required:
    [event_id, timestamp, user_id, action, entity_type, entity_id]
  properties:
    event_id:
      type: string
    timestamp:
      type: string
      format: date-time
    user_id:
      type: string
    action:
      type: string
    entity_type:
      type: string
    entity_id:
      type: string
    details:
      type: object
      additionalProperties: true

VulnerabilityFixture:
  type: object
  required: [cve_id, affected_purls]
  properties:
    cve_id:
      type: string
    affected_purls:
      type: array
      items:
        type: string

Error:
  type: object
  required: [error]
  properties:
```

```
    error:
      type: object
      required: [code, message]
      properties:
        code:
          type: string
        message:
          type: string
        details:
          type: object
          additionalProperties: true
```

Q5.2 Data schemas and examples
1 Point

Define the fields, types, required/optional status, and allowed values or constraints for the inputs and outputs. Specify units and formats where relevant. Include at least one valid request/response example for each core operation and one invalid-input example with its expected error. Cross-reference definitions in the contract rather than duplicating them.

General Data Rules:
-All request and response bodies use JSON unless otherwise stated.
-Dates use YYYY-MM-DD. Timestamps use ISO 8601 UTC format with a trailing Z (e.g., 2026-09-26T18:00:00Z).
-System-generated identifiers (release_id, incident_id, event_id) are nonempty strings and should be treated as opaque values by clients.
-For state-changing operations, the request must contain the X-User-Id HTTP header (a nonempty string).
-A release is considered supported through the end of its support_end_date and becomes unsupported on the following UTC calendar day.
1. Register a release (POST /api/v1/releases)
-Input (ReleaseCreate schema):
-product (string, required): Nonempty product name
-version (string, required): Nonempty version identifier
-support_end_date (string/date, required): YYYY-MM-DD
-Output (Release schema): Includes the input fields plus release_id (string, system-generated) and support_status (string: "supported" or "unsupported").
-Valid Request:

```
{
  "product": "ExampleRouter",
  "version": "2.4.1",
  "support_end_date": "2027-12-31"
}
```

-Valid Response (201 Created):

```
{
  "release_id": "rel-001",
  "product": "ExampleRouter",
  "version": "2.4.1",
  "support_end_date": "2027-12-31",
  "support_status": "supported"
}
```

-Invalid Request Example: A release request that omits the required support_end_date.
-Expected Error Response (400 Bad Request): The request must not create a release.

```
{
  "error": {
    "code": "INVALID_REQUEST",
    "message": "support_end_date is required"
  }
}
```

2. Submit an SBOM (PUT /api/v1/releases/{releaseId}/sbom)
-Constraints: The request must be a valid CycloneDX JSON document. Components are stored against the release identified by releaseId.

Valid Request:

```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.6",
  "version": 1,
  "components": [
    {
      "type": "library",
      "name": "example-library",
      "version": "3.2.0",
      "purl": "pkg:npm/example-library@3.2.0"
    }
  ]
}
```

-Valid Response (200 OK):

```
{
  "release_id": "rel-001",
  "component_count": 1
}
```

3. Retrieve release components (GET /api/v1/releases/{releaseId}/components)
-Constraints: A registered release with no stored SBOM returns an empty components array. An unknown release returns 404.
-Valid Response (200 OK):

```
{
  "release_id": "rel-001",
  "components": [
    {
      "name": "example-library",
      "version": "3.2.0",
      "purl": "pkg:npm/example-library@3.2.0"
    }
  ]
```

}

4. Query affected releases (GET /api/v1/vulnerabilities/{cveId}/affected-releases)
-Constraints: cveId must follow the format CVE-YYYY-NNNN... A known CVE that affects no stored releases returns 200 with empty supported and unsupported arrays. An unknown CVE returns 404 (CVE_UNKNOWN). An unavailable vulnerability source returns 503.
-Valid Request: GET /api/v1/vulnerabilities/CVE-2026-12345/affected-releases
-Valid Response (200 OK):

```
{
  "cve_id": "CVE-2026-12345",
  "supported": [
    {
      "release_id": "rel-001",
      "product": "ExampleRouter",
      "version": "2.4.1",
      "support_end_date": "2027-12-31",
      "matched_components": [
        {
          "name": "example-library",
          "version": "3.2.0",
          "purl": "pkg:npm/example-library@3.2.0"
        }
      ]
    }
  ],
  "unsupported": []
}
```

5. Create an incident (POST /api/v1/incidents)
-Input (IncidentCreate schema):
-cve_id (string/null, optional): CVE-YYYY-NNNN... format
-awareness_at (string/datetime, required): ISO 8601 UTC
-classification (string, required): "mandatory" or "voluntary"
-rationale (string, required): Nonempty explanation
-Valid Request:

```
{
 "cve_id": "CVE-2026-12345",
  "awareness_at": "2026-09-26T18:00:00Z",
  "classification": "mandatory",
  "rationale": "Confirmed active exploitation affecting a supported release."
}
```

Valid Response (201 Created): (For mandatory cases, the first two deadlines are calculated as awareness_at + 24 hours and awareness_at + 72 hours).

```
{
  "incident_id": "inc-001",
  "cve_id": "CVE-2026-12345",
  "awareness_at": "2026-09-26T18:00:00Z",
  "created_by": "engineer-17",
  "classification": "mandatory",
  "rationale": "Confirmed active exploitation affecting a supported release.",
  "fix_available_at": null,
  "deadlines": {
    "early_warning": {
      "deadline_at": "2026-09-27T18:00:00Z",
      "submitted_at": null,
      "status": "pending"
    },
    "vulnerability_notification": {
      "deadline_at": "2026-09-29T18:00:00Z",
      "submitted_at": null,
      "status": "pending"
    },
    "final_report": {
      "deadline_at": null,
      "submitted_at": null,
      "status": "not_applicable"
    }
  }
}
```


6. Record corrective-measure availability (PATCH /api/v1/incidents/{incidentId})
-Valid Request: {"fix_available_at": "2026-09-30T12:00:00Z"}
-Valid Response (200 OK): Uses the existing Incident schema, but the final deadline updates to:


```
{
  "final_report": {
    "deadline_at": "2026-10-14T12:00:00Z",
    "submitted_at": null,
    "status": "pending"
  }
}
```


7. Record a report submission (POST /api/v1/incidents/{incidentId}/submissions)
-Valid Request: {"stage": "early_warning", "submitted_at": "2026-09-27T17:45:00Z"}
-Valid Response (201 Created): Uses the Incident schema. A submission at or before its deadline changes the status to "met":

{
  "early_warning": {
    "deadline_at": "2026-09-27T18:00:00Z",
    "submitted_at": "2026-09-27T17:45:00Z",
    "status": "met"
  }
}

8. Store and generate report data (PUT .../report-data/{stage} and GET .../reports/{stage})
-Constraints: Partial input is allowed because the system must identify required fields that are still missing.
-Valid Report-Data PUT Request:
{
  "product_information": "ExampleRouter 2.4.1",
  "exploit_nature": "Remote exploitation of vulnerable library",
  "vulnerability_nature": "Input validation vulnerability",
  "corrective_measures_taken": "Patched library version prepared",
  "user_mitigations": "Disable affected feature until update"
}
Generated Report GET Response (200 OK):
{
  "incident_id": "inc-001",
  "stage": "vulnerability_notification",
  "generated_at": "2026-09-27T19:00:00Z",
  "data": {
    "product_information": "ExampleRouter 2.4.1",
    "exploit_nature": "Remote exploitation of vulnerable library",
    "vulnerability_nature": "Input validation vulnerability",
    "corrective_measures_taken": "Patched library version prepared",
    "user_mitigations": "Disable affected feature until update"
  },
  "missing_fields": []
}

9. Retrieve audit logs (GET /api/v1/audit-logs)
-Constraints: Returned chronologically. Attempts to modify or delete return 405 Method Not Allowed.
-Valid Response (200 OK):
{
  "events": [
    {
      "event_id": "evt-001",
      "timestamp": "2026-09-26T18:00:05Z",
      "user_id": "engineer-17",
      "action": "INCIDENT_CREATED",

```
      "entity_type": "incident",
      "entity_id": "inc-001",
      "details": {
        "classification": "mandatory"
      }
    }
  ]
}
```


Q5.3 Repeatable test setup
1 Point

(3-5 sentences): Explain how tests initialize or reset data, obtain any required authentication, and run against a selected team's base URL. Identify any shared fixtures, external-service assumptions, or ordering constraints. If the service is stateless or needs no authentication/reset, explicitly say so. Keep internal architectures and implementation technologies independent.

Before each automated test run, the evaluator will select the target team’s base URL and call POST /api/v1/test/reset to clear releases, SBOMs, incidents, report data, and audit logs so every test starts from the same state. No authentication is required for the common test interface; state-changing requests only include the required X-User-Id header so actions can be attributed in the audit log. Tests will use the pod’s shared fixtures, including the same sample releases, CycloneDX SBOMs, vulnerability records, incident cases, and controlled UTC clock, and may use the test vulnerability-source endpoint to simulate available or unavailable external vulnerability data. Tests that depend on existing state will create resources in the required order, such as registering a release before uploading its SBOM and creating an incident before recording submissions, while each team remains free to use any internal architecture or implementation technology

Q6 Milestones for Phases 3-5

4 Points

Shared work - complete with your pod.

For each phase, identify the capability to deliver, artifacts to produce or update, uncertainty to reduce, demonstration of completion, and automated validation to add. Use requirement and acceptance-criterion identifiers. Each milestone should produce a deployable, testable increment. The instructors will review the plan and may require scope or sequencing changes.

Q6.1 Phase 3

1 Point

Complete each field with a short statement.

Capability to deliver: Release registration, CycloneDX SBOM ingestion, and baseline testability.
Artifacts to produce or update: Database schemas; endpoints for /releases, /releases/{releaseId}/sbom, /test/reset, and /test/clock.
Uncertainty to reduce: The technical risk of parsing nested CycloneDX JSON files and correctly normalizing them into relational tables without data loss.
Demonstration of completion: Successfully deploying the API to a network-accessible server, registering a release, and persisting the pod's shared test SBOM payload.
Automated validation to add: AC-01, AC-02, and AC-03 (validating REQ-01 and REQ-02).

Q6.2 Phase 4
1 Point

Complete each field with a short statement.

Capability to deliver: Vulnerability querying with fault injection and incident deadline tracking.
Artifacts to produce or update: Endpoints for /vulnerabilities/{cveId}/affected-releases, /test/vulnerability-source, and /incidents; implementation of temporal deadline math.
Uncertainty to reduce: The risk of timezone drift in UTC deadline computation and ensuring reliable behavior when external vulnerability sources are unreachable.
Demonstration of completion: Utilizing the mock test clock to open a mandatory incident, verifying computed 24h/72h deadlines, and querying a CVE to retrieve affected product versions.
Automated validation to add: AC-04, AC-05, AC-06, and AC-07 (validating REQ-03, REQ-04, REQ-05, and REQ-06).

Q6.3 Phase 5
1 Point

Complete each field with a short statement.

Capability to deliver: CRA compliance report generation, immutable audit logging, and cross-team test execution.
Artifacts to produce or update: Endpoints for /incidents/{incidentId}/reports/{stage} and /audit-logs; final cross-team testing scripts.
Uncertainty to reduce: Guaranteeing audit log immutability and ensuring complete data persistence across server process restarts.
Demonstration of completion: Retrieving a formatted JSON stage report and successfully running the automated test suite against podmates' live deployed API URLs.
Automated validation to add: AC-08, AC-09, and AC-10 (validating REQ-07 and REQ-08).

Q6.4 Sequencing rationale
1 Point

(4-5 sentences): Explain why this order is feasible and how early milestones reduce risks or establish dependencies for later work. Identify any scope reduction the pod would make if progress falls behind.

This order is highly feasible because it builds the system logically from the foundational data layer outward. We must establish reliable SBOM storage and release tracking (Phase 3) before we can build the CVE querying logic that relies on mapping those exact components (Phase 4). Similarly, the core incident state machine and deadline calculations (Phase 4) must function correctly before we can aggregate that state into compliance reports or test the immutable audit log (Phase 5). This sequencing reduces early technical risks by proving we can parse the massive CycloneDX file structure and inject a mock test clock before attempting complex timeline math. If progress falls behind, the pod would reduce scope by dropping the voluntary incident classification logic (REQ-05) to focus strictly on mandatory deadlines, or by stripping out the unreachable-source fault-injection (REQ-04, AC-05) to simplify the testing infrastructure.

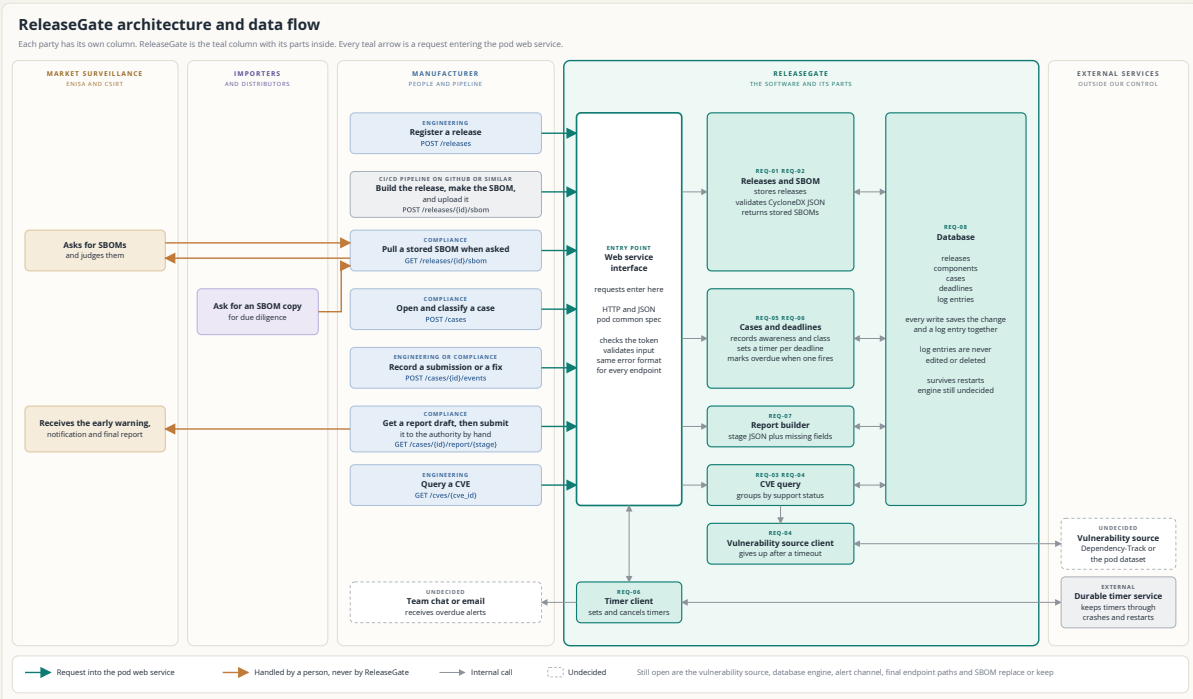
Q7 Team System Design
4 Points

Individual team work - complete Sections 7-9 within your team.

Q7.1 Architecture and data flow
1 Point

Provide a rough sketch showing your team's current idea of how the system could work. Label the main parts, show how information might move between them, and indicate where requests through the pod's common web-service interface would enter the system. Add a brief explanation of the approach and identify anything still undecided. A clearly labeled diagram is sufficient.

▾ 7.1.svg Download



Q7.2 Design decisions
1 Point

(two decisions, 3-4 sentences each): For each consequential design decision, state your choice, an alternative considered, and the tradeoff that led to your choice. Reference the requirement or quality attribute affected. Your initial design may evolve as you learn.

Decision 1: Compute deadlines on read instead of using a timer service.
We will store the awareness, fix-available, and submission timestamps for each incident and compute every deadline's pending, met, or overdue status on read against the injectable reference clock, instead of setting timers in a durable workflow service such as Temporal [7], which Phase 1 considered. Computing on read makes the Article 14(2) deadlines [1] behind REQ-06 follow the pod's test clock exactly, so AC-07's 0-second tolerance holds when another team advances the clock, and the stored timestamps survive restarts by construction for AC-10. A timer service would send alerts the moment a deadline passes, but it runs on wall-clock time, could disagree with our database after a crash, and adds a second system to operate. We give up instant alerts, which REQ-06 does not require; if reminders are needed later, a job that polls every few minutes is enough for deadlines of 24 hours to 14 days.

Decision 2: Reuse Dependency-Track for matching, behind our own release registry.
We will keep our own release and component tables in SQLite [11] and upload each release's CycloneDX SBOM [8] to OWASP Dependency-Track [3] for vulnerability matching, instead of writing a custom matcher that queries OSV [9] directly against our components. Both teams chose Dependency-Track in Phase 1, and its mature SBOM analysis supports REQ-03's 100% recall target without us building a scanner. The cost is two datastores to keep in sync, plus an extra network hop that uses part of AC-04's 10-second budget, and because Dependency-Track matches against its own mirrored vulnerability data, a very recent CVE may be missed until the mirror updates. We accept this because cross-team tests use fixed vulnerability records, and we treat Dependency-Track being unreachable as REQ-04's "source unreachable" case, detected explicitly rather than returned as an empty list.

Q7.3 Operation and observability
1 Point

(3-5 sentences): Explain how your team plans to deploy and run the web service, manage persistent data and required secrets/access, and detect failures. Identify the logs, metrics, or other signals needed to diagnose incorrect behavior. If the system stores no persistent data, state that.

We will deploy the service as a Docker Compose stack on [host — a cloud VM with at least 8 GB RAM and 4 cores] with two containers: our FastAPI [10] application, with its SQLite [11] database file on a named persistent volume, and Dependency-Track with its own database volume, sized to its documented recommendation [2]. A nightly job copies the SQLite file using SQLite's online backup and keeps a week of copies, and the test-reset and clock endpoints are enabled only in the evaluation environment, which holds synthetic data. The Dependency-Track API key lives in an untracked .env file on the host, and only our API port is publicly exposed. We detect failures through a /health endpoint that checks the database is writable and Dependency-Track is reachable, and an external uptime monitor polls it. Every request writes a structured JSON log line with the request ID, X-User-Id, endpoint, status code, latency, injected clock value, and any Dependency-Track error. We track per-endpoint error rates, p95 latency against AC-04's 10-second limit, how often Dependency-Track is unreachable, SBOM rejections by reason, and overdue-deadline counts; together with the audit log, these signals let us reconstruct which input, user, and clock value produced a wrong result.

Q7.4 Validation strategy
1 Point

(3-4 sentences): Describe a small prototype, experiment, or test your team could try first to validate its components and integrated system against Section 4. Identify the static checks, dynamic tests, or other analyses appropriate to the design and when they will run. Explain how failures will be investigated and regressions prevented.

Our first prototype is a thin slice that registers one release, uploads the pod's shared 25-component CycloneDX SBOM [8] to both SQLite and Dependency-Track, and queries one seeded CVE, testing the riskiest integration (AC-02, AC-04) before any incident logic exists. On every push, GitHub Actions runs linting, type checking, and a contract check of our generated OpenAPI document against the Section 5 specification. Every pull request must also pass unit tests for the Article 14(2) deadline math [1] using the injected clock, covering submissions before, at, and after each deadline plus month and year rollovers, as well as the AC-01–AC-10 HTTP suite against a freshly reset stack with the mocked vulnerability source; at the end of each phase, we also run the suite against the other team's deployed URL. When a test fails, we replay it from the logged request ID and clock value, fix the cause, and add that case as a permanent regression test.

Q8 Build Buy and Reuse
2 Points

Individual team work - complete within your team.

Identify at least one major capability (a capability or dependency such that its failure or replacement would affect the outcome of a requirement, security, cross-team testing, delivery effort, or schedule), and explain how your team will obtain each of them.
Consider building it, using a library/framework, integrating an existing system, using a hosted service, or delegating suitable work to a GenAI service. Refer to Phase 1 research where it applies.

Q8.1 Capability decisions
1 Point

For each major capability or dependency, identify the selected approach and candidate tool/service, explain why it fits, and describe the integration work your team must do. Address the cost, licensing, security, reliability, and maintenance implications that matter to the decision. Cite technical documentation or other supporting evidence.

Vulnerability analysis: Dependency-Track (REQ-02–REQ-04). We will reuse OWASP Dependency-Track to match CycloneDX SBOMs [8] against known vulnerabilities, as both teams chose in Phase 1 [3]. We will map each release to a Dependency-Track project, upload SBOMs through its REST API, group findings by support status, and convert unknown CVEs and connection failures into REQ-04 errors. It is free under Apache 2.0 [3], but it needs 8 GB RAM and 4 cores recommended [2], which sets our hosting cost. We will keep its port private, limit its API key, and pin its version. Because it is a single point of failure for REQ-03, tests use fixed vulnerability records, and Risk 1 (Section 9.1) defines a local fallback.

Web service and deadlines: FastAPI (REQ-01–REQ-07). We will build the service in FastAPI, which is MIT-licensed and generates OpenAPI documentation we can check against Section 5 [10]. We will write the endpoints, SBOM validation, deadline math, and JSON reports ourselves. Deadlines are computed from stored timestamps (Section 7.2), so we skip Temporal [7]. We also skip TheHive [4], which needs a license and lacks the Article 14(2) deadline logic [1]. We will pin dependencies and limit the unauthenticated test endpoints to an evaluation environment with synthetic data.

Storage and audit history: SQLite (REQ-08). We will use SQLite, which is public domain and needs no database server [11], instead of PostgreSQL [6]. Each change and its audit entry are written in one transaction, and database triggers block audit edits, as checked by AC-09 and AC-10. SQLite allows one writer at a time [12], which is fine for a single-instance prototype. The file sits on a restricted persistent volume with nightly backups, but this cannot stop a host administrator from replacing it.

Q8.2 GenAI use
1 Point

(3-4 sentences): State how your team expects to use GenAI during engineering and whether AI is part of the running system. Identify available tools/licenses, work delegated, and how outputs will be checked. Describe any effect on data handling, cost, or reliability. If you do not plan to use GenAI for a category, state that.

We plan to use coding assistants such as Claude Code etc. since we collectively have subscriptions for ChatGPT Plus, Gemini Pro, and Claude Pro. We will use them for boilerplate code, test drafts, and documentation, then review the output and run the shared acceptance tests, with particular attention to deadlines and audit logging. AI will not be part of the running system, so it may reduce development time but will not affect runtime costs or make test results depend on AI-generated answers.

Q9 Major Risks and Investigation Plan

2 Points

Individual team work - complete within your team.

Identify the uncertainties most likely to prevent successful delivery. Consider technical feasibility, regulatory ambiguity, integration, security/privacy, AI reliability, performance, test data, validation, and external dependencies.

Q9.1 Priority risks
1 Point

Describe two major risks, ordered by priority. For each, state the uncertainty , the effect if your team's assumptions are wrong, and a concrete investigation proposal. Identify who will investigate, when it must be resolved, and the evidence that would support a decision. Link to a milestone in Section 6 and include a fallback if the risk cannot be reduced

Risk 1 (highest): Dependency-Track matching, replacement, and fault behavior

Uncertainty: We assume OWASP Dependency-Track [2, 3] can analyze uploaded CycloneDX SBOMs [8] and answer CVE queries for REQ-03 and REQ-04 under the shared Section 5 test rules: simulating an unreachable source, resetting all data between runs, and fully replacing a release's components when a newer SBOM is uploaded (Section 3.3).

Effect if assumptions are wrong: Dependency-Track analyzes uploads in the background against its own vulnerability data. If analysis lags behind the upload, if component identifiers in the shared data don't match its format, if replacement leaves stale components, or if it cannot distinguish an unknown CVE from one with no matches, our system will return slow or wrong results. That would fail AC-04 and AC-05 and delay the Phase 4 milestone (Section 6.2).

Investigation proposal, owner, and deadline: We will set up a test instance [2] before the Phase 3 milestone (Section 6.1), upload the shared 25-component SBOM [8], replace it with a second SBOM for the same release, reset the system, and run the five seeded CVE queries.

Decision evidence: Logs showing that queries return within AC-04's 10-second limit immediately after upload, that no stale components remain after replacement or reset, that recall is 100% on the seeded manifest, and that an unknown CVE gets a distinct response — all confirmed before Phase 4 begins.

Fallback: Match vulnerabilities in our own application against the components already stored in SQLite [11], using a local table loaded from the shared test data or a saved OSV feed [9]. This also simplifies fault injection for AC-05.

Risk 2: Ambiguity over when "awareness" starts the Article 14 clock

Uncertainty: Article 14(2) measures deadlines from when the manufacturer becomes aware of an actively exploited vulnerability [1], but the pod has not

agreed whether that means the raw alert time or engineer-verified confirmation.

Effect if assumptions are wrong: The teams would compute different expected deadlines from the same fixture, so AC-06 and AC-07 would disagree in cross-team testing, and the Phase 4 deadline work (Section 6.2) would need rework.

Investigation proposal, owner, and deadline: We will review Article 14 [1] and any ENISA or Commission guidance on awareness, then propose a rule to the pod before the shared dataset is frozen at the Phase 3 milestone (Section 6.1).

Decision evidence: Cited guidance text and a pod-agreed rule recorded in the specification.

Fallback: Keep awareness_time as a user-supplied field, as REQ-05 already does, so the system never infers it and tests stay deterministic regardless of the legal interpretation.

Q9.2 Risks requiring pod agreement
1 Point

(2-3 sentences): From Section 9.1, identify the risks that could require a change to the shared scope, requirements, acceptance criteria, or interface. Explain what the pod needs to decide and when. If none is currently identified, state that. Keep the common specification consistent when decisions change.

From Risk 1 and the open assumption in Section 3.3, the pod must decide before the Phase 3 milestone whether uploading a new SBOM for an existing release fully replaces its component list or keeps a version history, since this affects REQ-02, AC-02, and the Section 5 upload behavior. Before Phase 4, the pod must also agree whether REQ-03 and AC-04 match on full package identifiers or only on component names and versions, so that both teams return identical affected releases from the shared dataset. From Risk 2, the pod must confirm, before the shared dataset is frozen in Phase 3, that awareness_time remains a user-supplied field, so that no legal interpretation of "awareness" can change the expected deadlines in AC-06 or AC-07.

Q10 References
1 Point

Include 5–8 distinct, relevant sources. Number sources in the order they first appear in your report, and reuse the same number whenever you cite that source again.
Place citations immediately after the supported claim and identify a section or provision when needed:
Regulatory claim: “The obligation includes documenting product components [1, Annex I, Part II(1)].”
Interface description: “OpenAPI provides a structured description of an HTTP API [1].”
Protocol behavior: “A 400 response indicates a perceived client error [2, Section 15.5.1].”

Format each reference as follows:
[Number] Author or issuing organization. “Title.” Publication date or version, if available. URL. Accessed: YYYY-MM-DD.

Example:
[1] European Parliament and Council of the European Union. “Regulation (EU) 2024/2847 (Cyber Resilience Act).” October 23, 2024. https://eur-lex.europa.eu/eli/reg/2024/2847/oj. Accessed: 2026-09-08.

References
[1] European Parliament and Council of the European Union. "Regulation (EU) 2024/2847 (Cyber Resilience Act)." October 23, 2024. https://eur-lex.europa.eu/eli/reg/2024/2847/oj. Accessed: 2026-09-26.
[2] OWASP Dependency-Track. "Deploying Docker Container." Dependency-Track Documentation, v4.14. https://docs.dependencytrack.org/getting-started/deploy-docker/. Accessed: 2026-09-26.
[3] OWASP Dependency-Track. "Dependency-Track." GitHub repository. https://github.com/DependencyTrack/dependency-track. Accessed: 2026-09-26.
[4] StrangeBee. "TheHive Licenses." TheHive Documentation. https://docs.strangebee.com/thehive/installation/licenses/about-licenses/. Accessed: 2026-09-26.
[5] Anchore. "Syft." GitHub repository. https://github.com/anchore/syft. Accessed: 2026-09-26.
[6] The PostgreSQL Global Development Group. "PostgreSQL License." https://www.postgresql.org/about/licence/. Accessed: 2026-09-26.
[7] Temporal Technologies. "Temporal." GitHub repository. https://github.com/temporalio/temporal. Accessed: 2026-09-26.
[8] OWASP CycloneDX. "CycloneDX Specification Overview." https://cyclonedx.org/specification/overview/. Accessed: 2026-09-26.
[9] Google Open Source Security Team. "OSV: Open Source Vulnerabilities." https://osv.dev/. Accessed: 2026-09-26.

[10] FastAPI. "FastAPI." GitHub repository. https://github.com/fastapi/fastapi. Accessed: 2026-09-27.

[11] SQLite. "SQLite Copyright." https://www.sqlite.org/copyright.html. Accessed: 2026-09-27.

[12] SQLite. "Appropriate Uses for SQLite." https://www.sqlite.org/whentouse.html. Accessed: 2026-09-27.